Asynchronous Programming

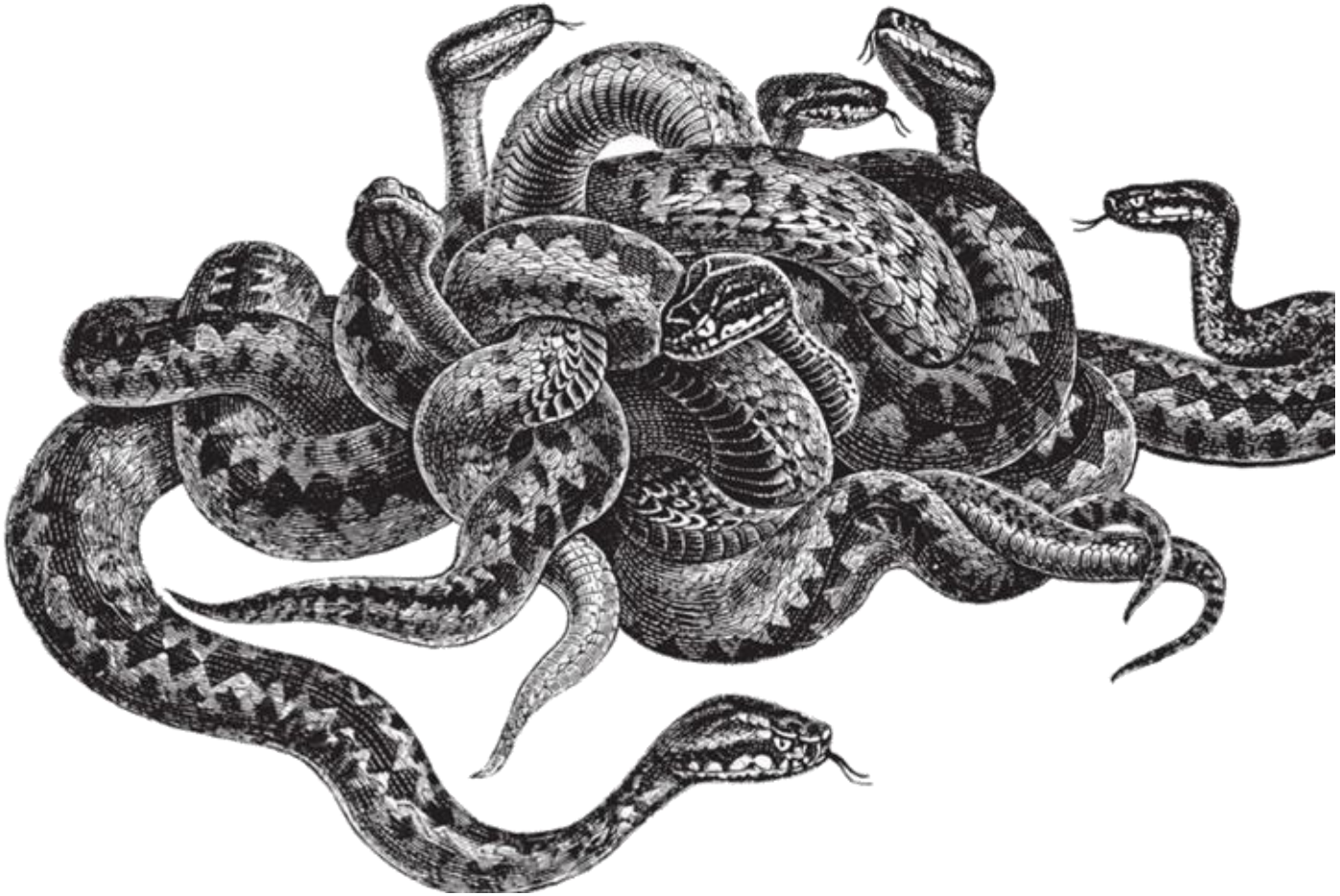


Image credit: "Twisted - Network Programming Essentials" by Abe Fettig

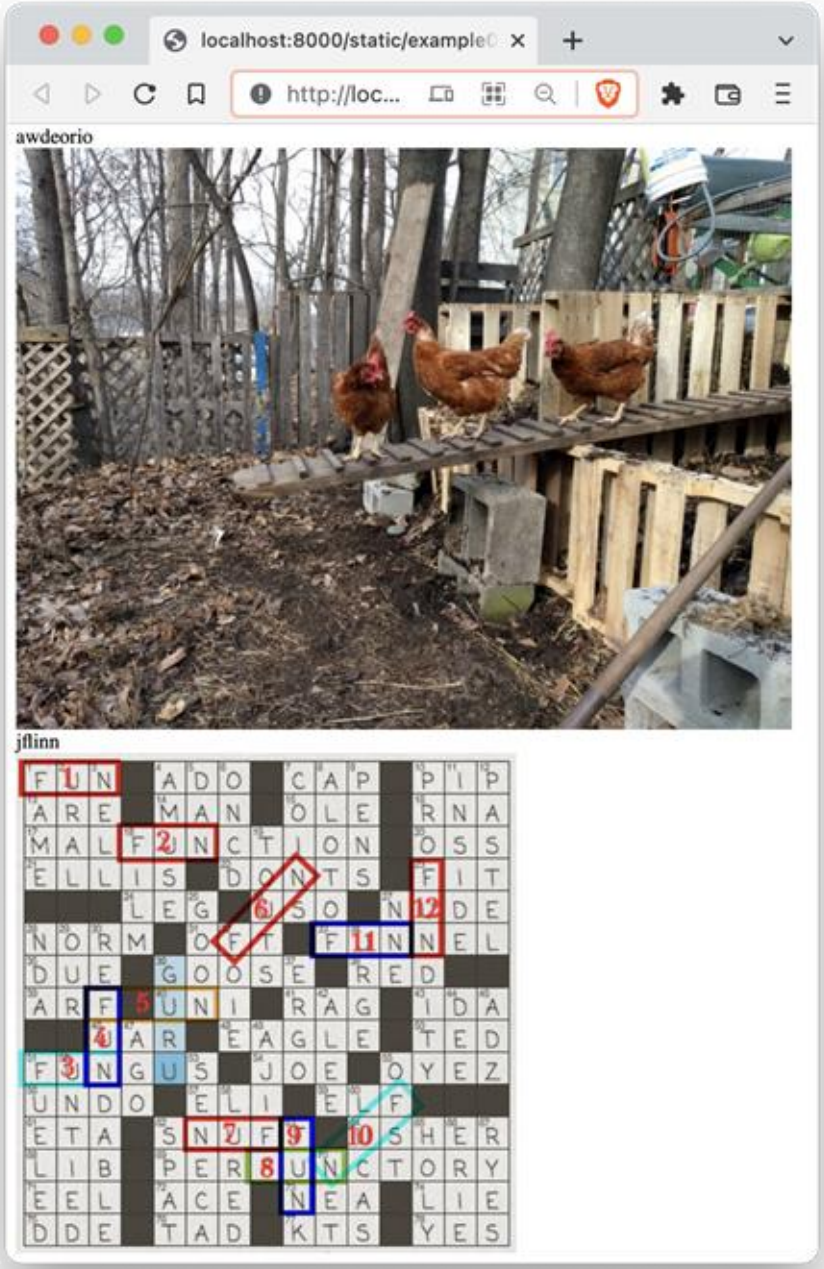
Some Slides by Andrew DeOrio

Agenda

- Synchronous requests
- Asynchronous requests
- Promise review
- Asynchronous requests with ordered page updates
- Closures
- Idiomatic asynchronous requests with Promise chains

Simplified Instagram

- We'll use a simplified Instagram clone for today's examples
- No buttons
- Post 1 is chickens
- Post 2 is crossword



HTML

- HTML contains an **entry point** for JavaScript and a **function**

```
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
     function showPosts() {
       // ...
     }
     showPosts();
    </script>
  </body>
</html>
```

Synchronous

```
function fetchJSONSync(url) {
  // Synchronous GET request to url,
  // decode JSON and return object.
}
```

- Synchronous: one task at a time

```
function showPosts() {
  post1 = fetchJSONSync("/api/v1/posts/1/");
  updateDOM(post1);

  post2 = fetchJSONSync("/api/v1/posts/2/");
  updateDOM(post2);
}
```

REST API

- Simplified REST API like Project 3's post detail route

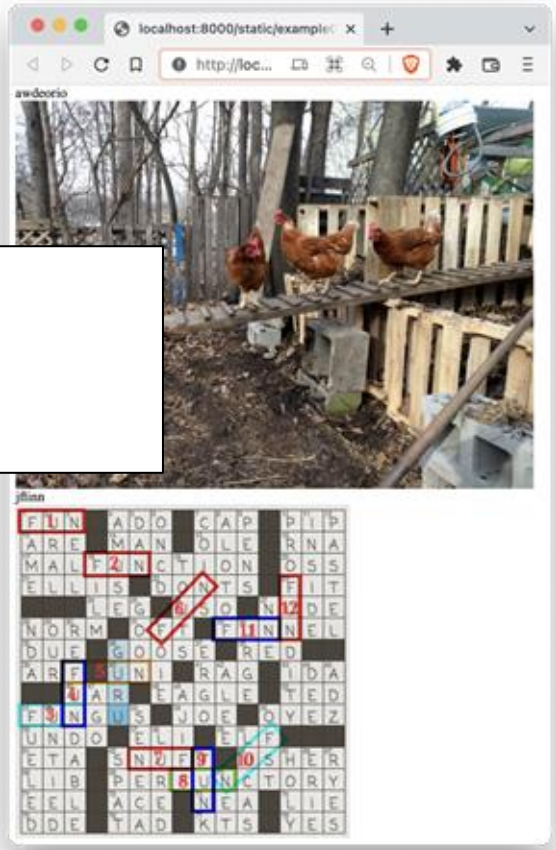
```
$ http localhost:8000/api/v1/posts/1/
{
    "imgUrl": "/uploads/122a7d27ca1d7420a1072f695d9290fad4501a41.jpg",
    "owner": "awdeorio",
}

$ http localhost:8000/api/v1/posts/2/
{
    "imgUrl": "/uploads/ad7790405c539894d25ab8dcf0b79eed3341e109.jpg",
    "owner": "jflinn",
}
```

JavaScript DOM manipulation

```
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
          ...
   </script>
  </body>
</html>
```

- Helper function appends DOM nodes for one post

```
// Append post to DOM.
function updateDOM(post) {
......
}
```



JavaScript DOM manipulation

```
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
          ...
   </script>
  </body>
</html>
```

- Helper function appends DOM nodes for one post

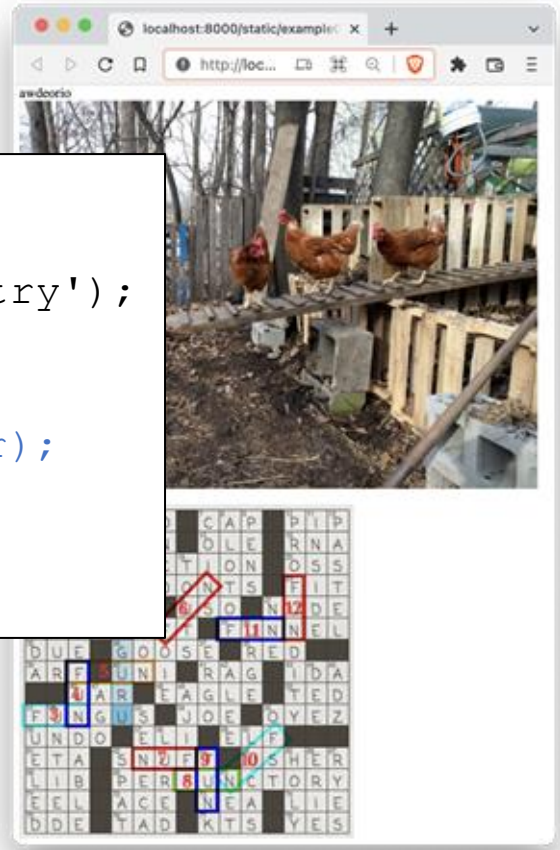
```
// Append post to DOM.
function updateDOM(post) {
  const entry = document.getElementById('JSEntry');
  const div = document.createElement('div');
  entry.appendChild(div);
……
}
```

JavaScript DOM manipulation

```
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
          ...
   </script>
  </body>
</html>
```

- Helper function appends DOM nodes for one post

```
// Append post to DOM.
function updateDOM(post) {
  const entry = document.getElementById('JSEntry');
  const div = document.createElement('div');
  entry.appendChild(div);
  const t = document.createTextNode(post.owner);
  div.appendChild(t);
……
}
```

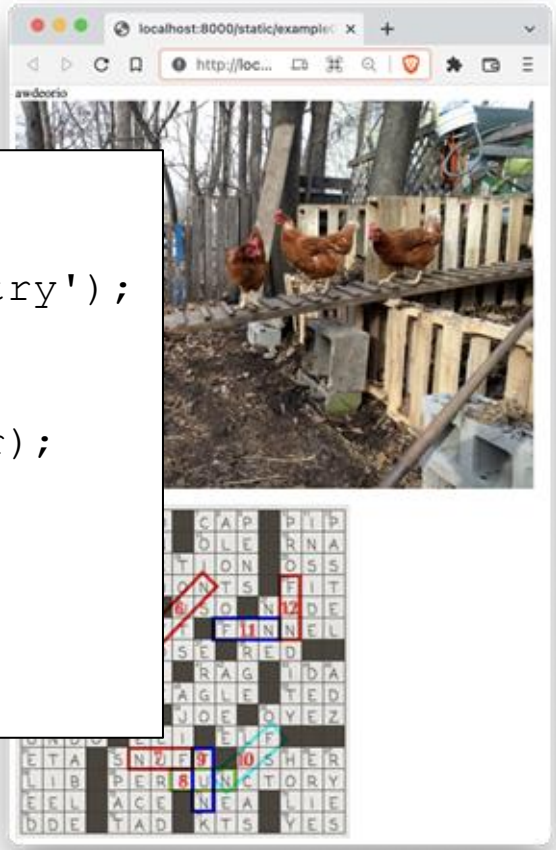


JavaScript DOM manipulation

```
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
          ...
   </script>
  </body>
</html>
```

- Helper function appends DOM nodes for one post

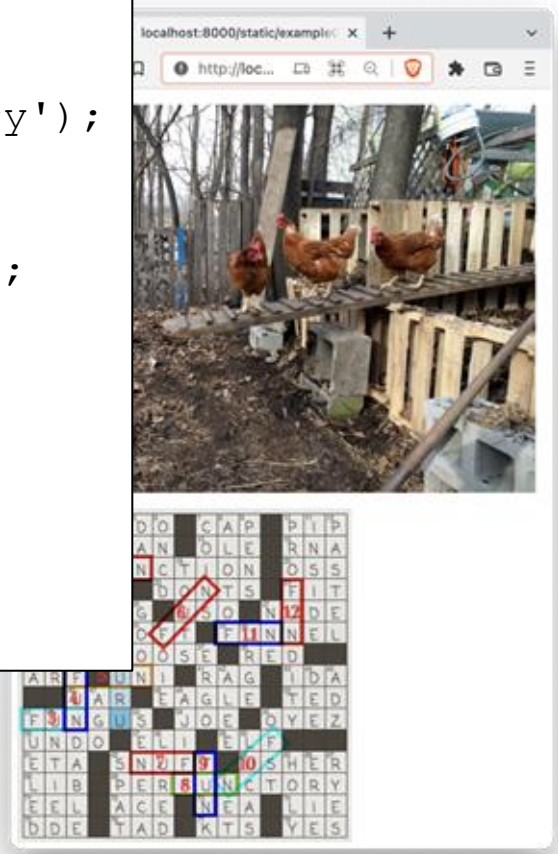
```
// Append post to DOM.
function updateDOM(post) {
  const entry = document.getElementById('JSEntry');
  const div = document.createElement('div');
  entry.appendChild(div);
  const t = document.createTextNode(post.owner);
  div.appendChild(t);
  const br = document.createElement('br');
  div.appendChild(br);
  ……
}
```



JavaScript DOM manipulation

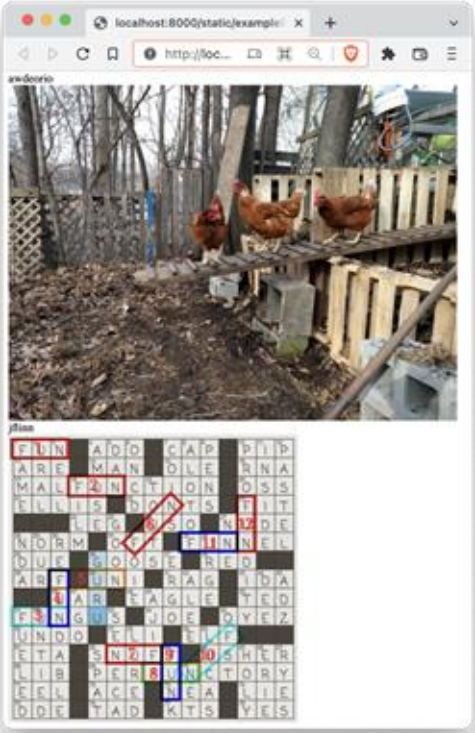
```html
<html>
  <body>
    <div id="JSEntry"></div>
    <script>
          ...
   </script>
  </body>
</html>
```

```javascript
// Append post to DOM.
function updateDOM(post) {
  const entry = document.getElementById('JSEntry');
  const div = document.createElement('div');
  entry.appendChild(div);
  const t = document.createTextNode(post.owner);
  div.appendChild(t);
  const br = document.createElement('br');
  div.appendChild(br);
  const img = document.createElement('img');
  img.setAttribute('src', post.imgUrl);
  div.appendChild(img);
}
```



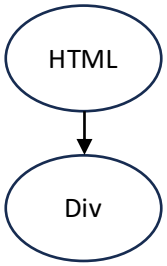
Memory diagram

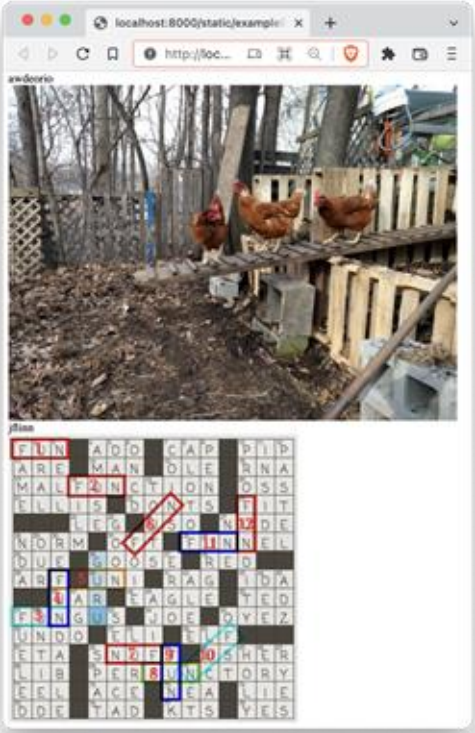
```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```



Memory diagram

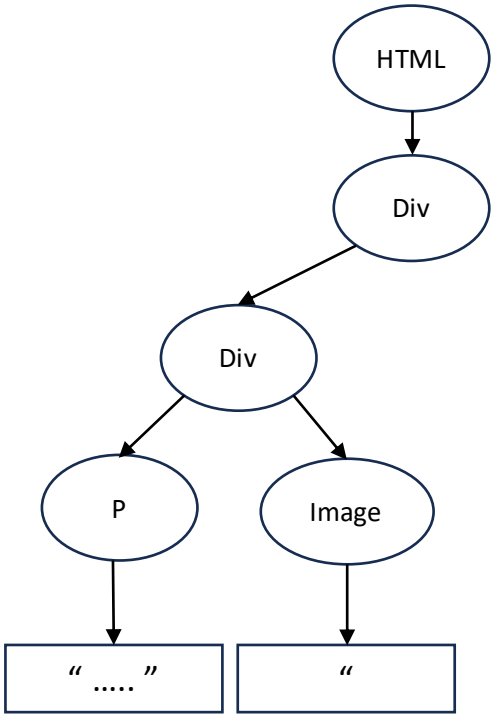
```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```

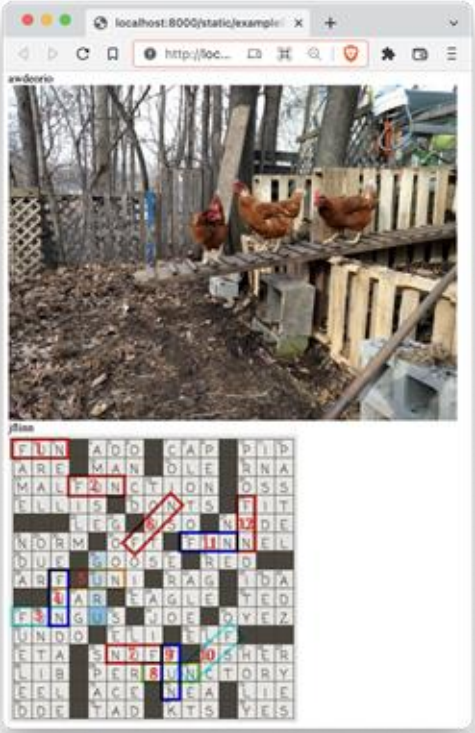




Memory diagram

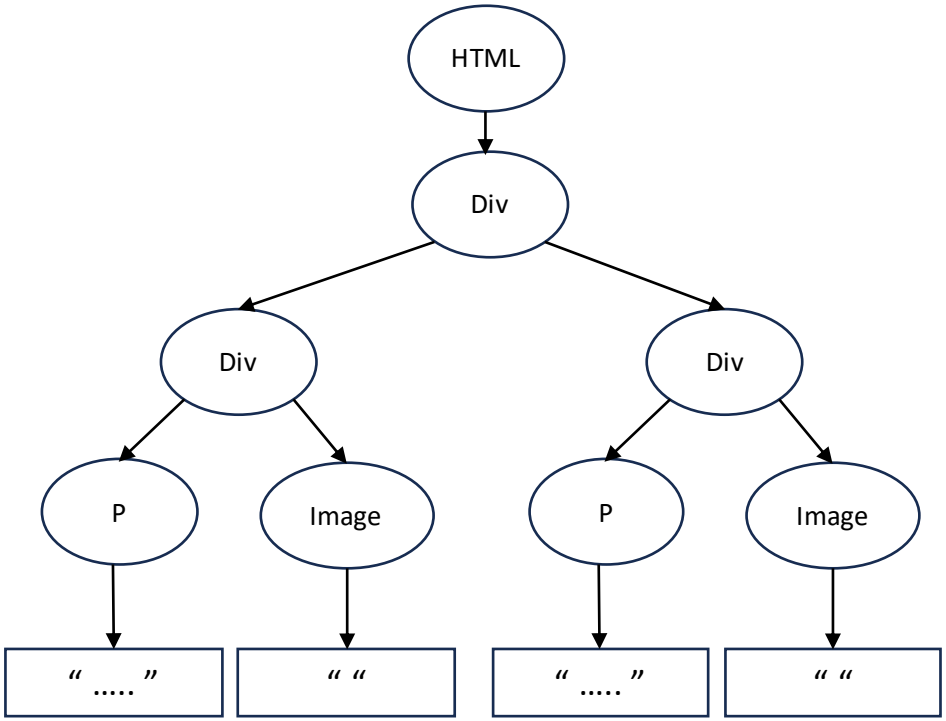
```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```

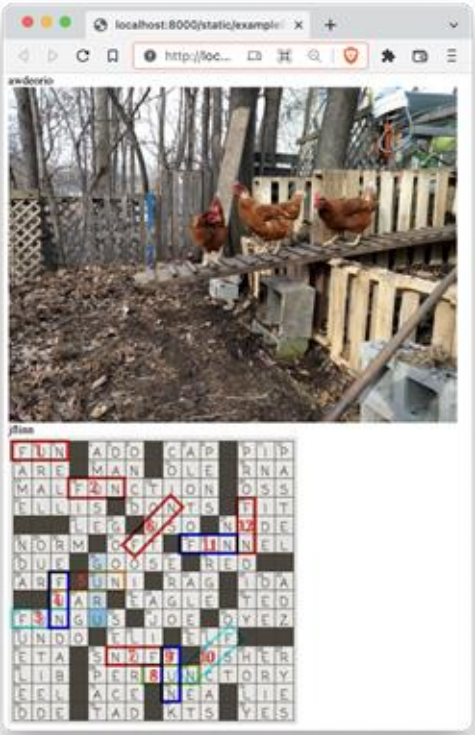




Memory diagram

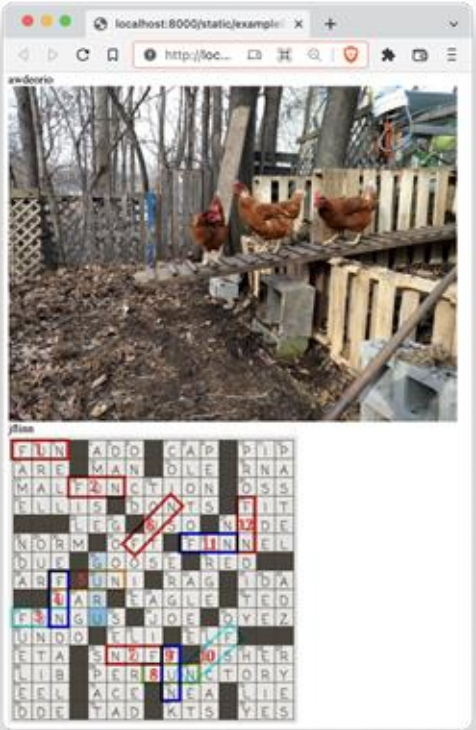
```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```





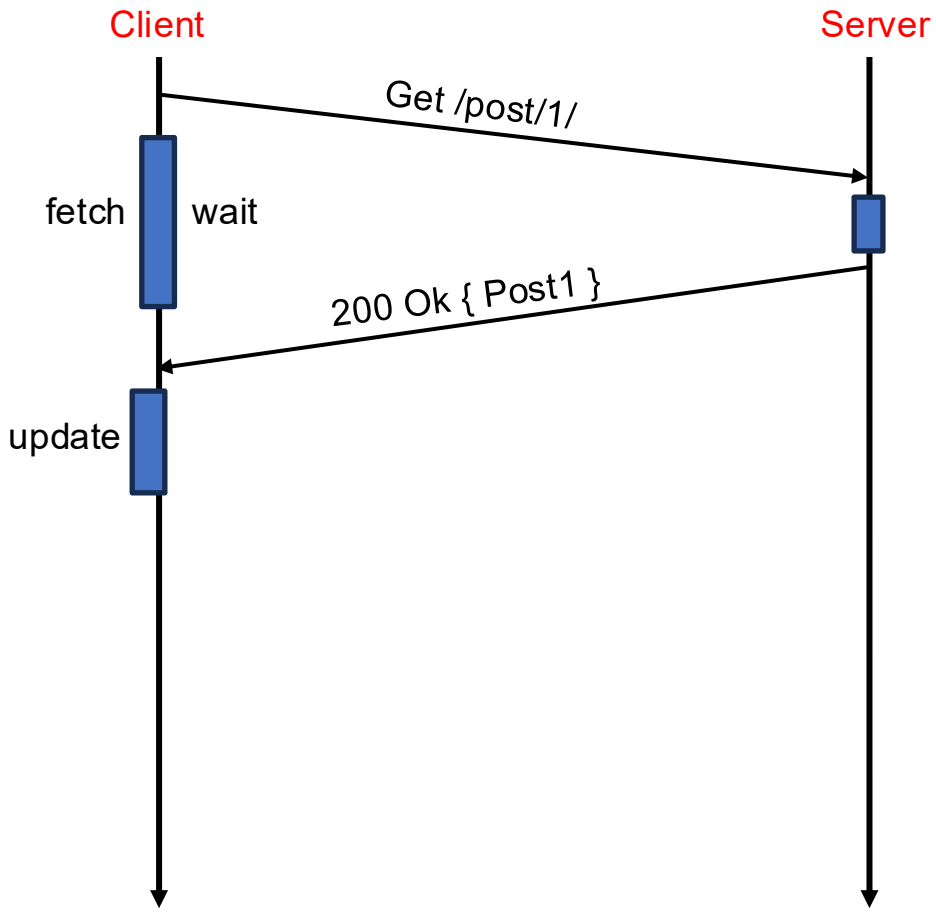
Timing diagram

```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```



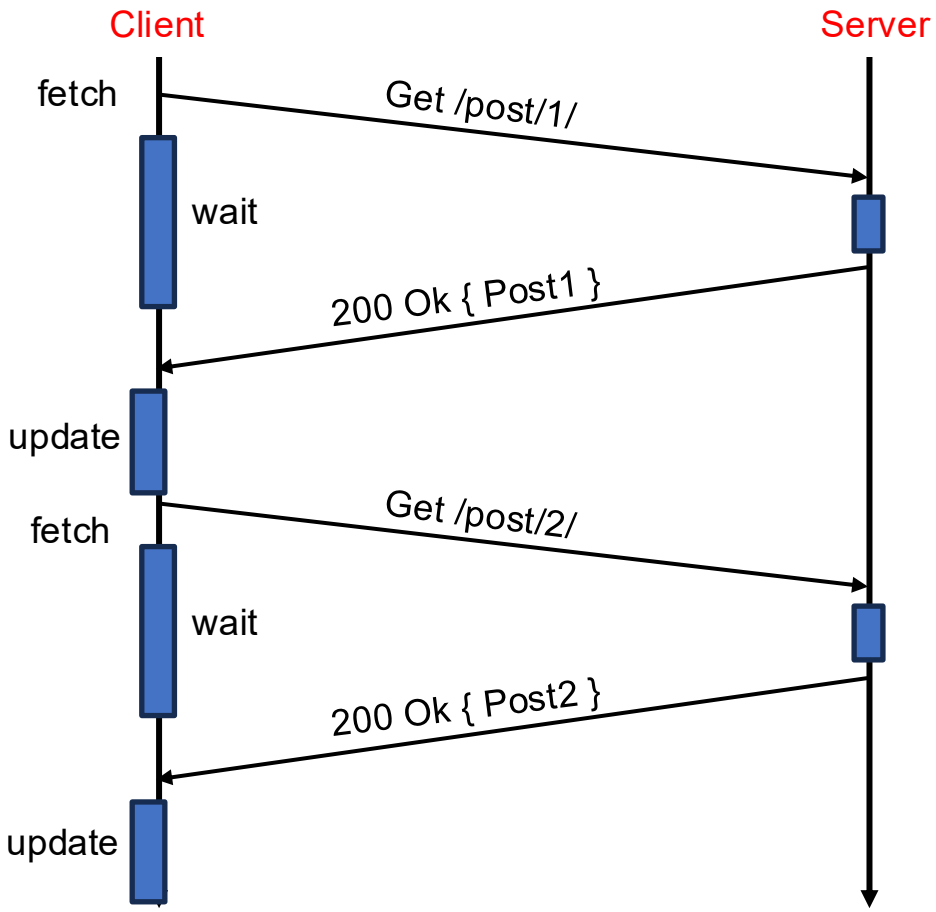
Timing diagram

```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```



Timing diagram

```
function showPosts() {
  post1 = fetchJSONSync(".../1/");
  updateDOM(post1);
  post2 = fetchJSONSync(".../2/");
  updateDOM(post2);
}
```



Agenda

- Synchronous requests
- **Asynchronous requests**
- Promise review
- Asynchronous requests with ordered page updates
- Closures
- Idiomatic asynchronous requests with Promise chains

Synchronous requests must wait

- **Problem**: synchronous requests are slow
- Can't send the second request until the first response has arrived

```
function showPosts() {
  post1 = fetchJSONSync("/api/v1/posts/1/");
  updateDOM(post1);

  post2 = fetchJSONSync("/api/v1/posts/2/");
  updateDOM(post2);
}
```

Solution: asynchronous requests

- **Problem**: synchronous requests are slow
- **Solution**: send both requests right away ***without*** *waiting for the response*

```
function showPosts() {
  const post_promise1 = fetchJSON("/api/v1/posts/1/")
  const post_promise2 = fetchJSON("/api/v1/posts/2/")

   // Wait for responses ...

  updateDOM(post1);
  updateDOM(post2);
}
```

- `fetchJSON` is a simplified version of the Fetch API

Asynchronous programming

- *Asynchronous programming*: interleave tasks
- Requests run in the background while the program keeps going.
- Programmer controls when tasks take turns

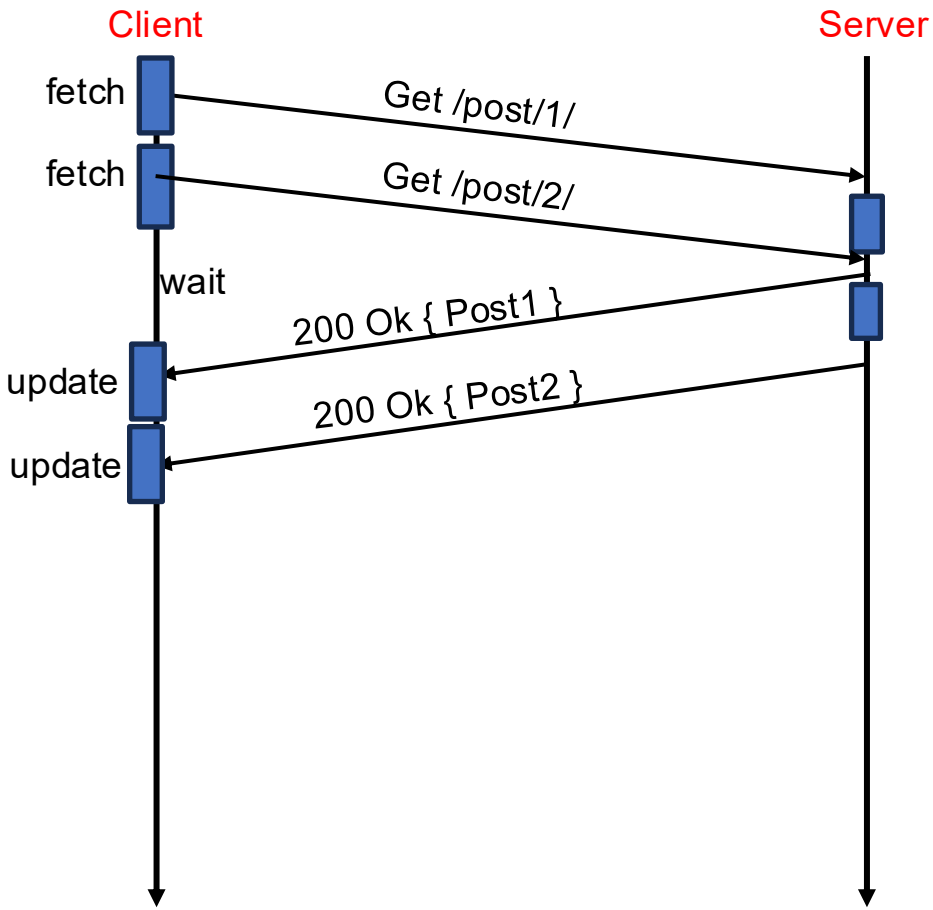
Async timing diagram

```
function showPosts() {
  const postPromise1 = fetchJSON(".../1/");
  const postPromise2 = fetchJSON(".../2/");
  // Wait for responses* ...
  updateDOM(post1);
  updateDOM(post2);
}
```


*This code is incomplete. We still need to connect `postPromise1` to `post1`.

Async timing diagram

```
function showPosts() {
  const postPromise1 = fetchJSON(".../1/");
  const postPromise2 = fetchJSON(".../2/");
  // Wait for responses* ...
  updateDOM(post1);
  updateDOM(post2);
}
```



*This code is incomplete. We still need to connect `postPromise1` to `post1`.

Solution: asynchronous requests

- **Problem**: If we send both requests right away, the response might not be ready yet. What should `fetchJSON()` return?

```
function showPosts() {
  const post_promise1 = fetchJSON("/api/v1/posts/1/")
  const post_promise2 = fetchJSON("/api/v1/posts/2/")

   // Wait for responses ...

  updateDOM(post1);
  updateDOM(post2);
}
```

Solution: asynchronous requests

- **Problem**: If we send both requests right away, the response might not be ready yet. What should `fetchJSON()` return?
- **Answer**: Promises that will eventually hold the post

```
function showPosts() {
  const post_promise1 = fetchJSON("/api/v1/posts/1/")
  const post_promise2 = fetchJSON("/api/v1/posts/2/")

   // Wait for responses ...

  updateDOM(post1);
  updateDOM(post2);
}
```

Review: Promises

- A *Promise* represents a value that will be available in the future
- A *Promise* controls the flow of asynchronous operations
- `fetchJSON()` sends a request
- `fetchJSON()` returns a Promise

```
function showPosts() {
  const postPromise1 = fetchJSON("/api/v1/posts/1/");
  const postPromise2 = fetchJSON("/api/v1/posts/2/");
  // Wait for responses ...
}
```

Review: Promise handler (callback)

- Promise *handler* runs when Promise is fulfilled
- **Handler is a function** provided by `.then()`

```
function showPosts() {
  const postPromise1 = fetchJSON("/api/v1/posts/1/");
  postPromise1.then(post => {
    updateDOM(post);
  });
}
```

Review: Refactor

- If we refactor, will the output change?
- No. The handler runs when the response arrives.

```
function showPosts() {
  const prom1 = fetchJSON("/api/v1/posts/1/");
  const prom2 = fetchJSON("/api/v1/posts/2/");

  prom1.then(post => {
    updateDOM(post);
  });
  prom2.then(post => {
    updateDOM(post);
  });
}
```

```
function showPosts() {
  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post);
  });

  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post);
  });
}
```

Agenda

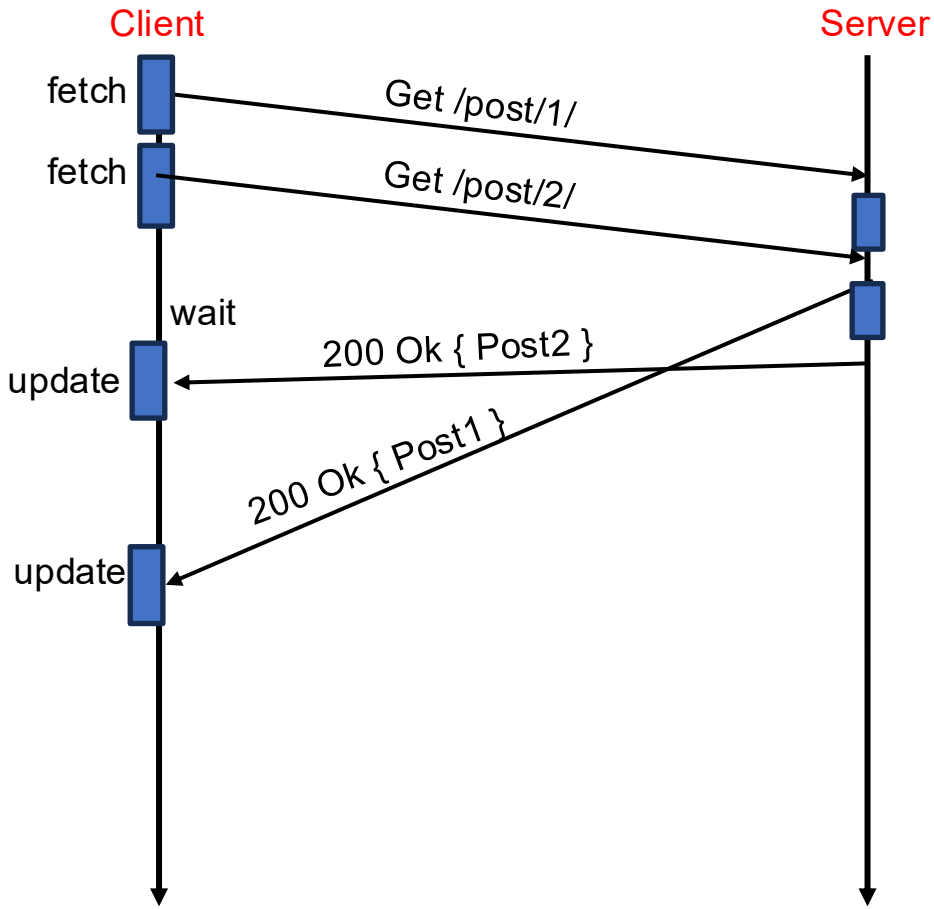
- Synchronous requests
- Asynchronous requests
- Promise review
- **Asynchronous requests with ordered page updates**
- Closures
- Idiomatic asynchronous requests with Promise chains

Problem with asynchronous requests

- What happens if responses arrive out of order?
  - E.g., varying network conditions
- Let's modify our REST API to randomly delay some responses
  - Some responses are **fast**, some are **slow**

```
INFO in app: Request /api/v1/posts/1/ received
INFO in app: Request /api/v1/posts/1/ delayed
INFO in app: Request /api/v1/posts/2/ received
INFO in app: Response /api/v1/posts/2/
INFO in app: Response /api/v1/posts/1/
```

Timing diagram with delayed response



```
function updateDOM(post) {
  // Append post to DOM.
}
```

Ordering problem

- Problem: different response order -> different page display order

Solution: ordered placeholders

- Problem: different response order -> different page display order
- Solution: placeholders

1. Create placeholder `div` nodes *in the right order*
2. Send requests
3. When response arrives, update the right placeholder

Placeholders

- Replace single DOM entry node with several placeholder nodes
- You could also do this with JavaScript

```
<html>
   <div id="JSEntry">Loading posts ...</div>
   <!-- ... -->
</html>
```

```
<html>
   <div id="JSEntry1">Loading post 1 ...</div>
   <div id="JSEntry2">Loading post 2 ...</div>
   <!-- ... -->
</html>
```

Aside: helper function

- Helper function updates DOM *at a specific div*

```
function updateDOM(post, div) {
  const t = document.createTextNode(post.owner);
  div.appendChild(t);
  const br = document.createElement('br');
  div.appendChild(br);
  const img = document.createElement('img');
  img.setAttribute('src', post.imgUrl);
  div.appendChild(img);
}
```

Placeholders

```
<html>
   <div id="JSEntry1">Loading post 1 ...</div>
   <div id="JSEntry2">Loading post 2 ...</div>
   <!-- ... -->
</html>
```

- When `response 1` arrives, callback updates first placeholder div
- When `response 2` arrives, callback updates second placeholder div
- Order of responses does not matter

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
  const div2 = document.getElementById('JSEntry2');

  // Send requests, update the right div
  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post, div1);
    });
  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post, div2);
  });
}
```

Ordered page from unordered responses

Page always looks like this, even if response 2 arrives first

Page never looks like this

Agenda

- Synchronous requests
- Asynchronous requests
- Promise review
- Asynchronous requests with ordered page updates
- **Closures**
- Idiomatic asynchronous requests with Promise chains

Inner functions

- An *inner function* is a function defined inside a function
- Our previous example used (anonymous) *inner functions*

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
  const div2 = document.getElementById('JSEntry2');

  // Send requests, update the right div
  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post, div1);
    });
  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post, div2);
  });
}
```

Access outer function's variable

- The *inner function* has access to the *outer function's* variables

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
  const div2 = document.getElementById('JSEntry2');

  // Send requests, update the right div
  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post, div1);
    });
  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post, div2);
  });
}
```

Outer function returns first

- The *outer function* returns first
- The *inner function* runs later

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
  const div2 = document.getElementById('JSEntry2');

  // Send requests, update the right div
  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post, div1);
    });
  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post, div2);
  });
}
```

Closure

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
   fetchJSON("/api/v1/posts/1/")
     .then(post => {
       updateDOM(post, div1);
     });
}
```

- An inner function has access to an outer function's variables
- Lexically scoped name binding
- This is called a **closure**

- What happens to the outer function variables when the outer function returns?
- The activation record sticks around

Agenda

- Synchronous requests
- Asynchronous requests
- Promise review
- Asynchronous requests with ordered page updates
- Closures
- **Idiomatic asynchronous requests with promise chains**

Simplifications

- Hardcoded HTML placeholders for 2 posts

```
<html>
   <div id="JSEntry1">Loading post 1 ...</div>
   <div id="JSEntry2">Loading post 2 ...</div>
   <!-- ... -->
</html>
```

Simplifications

- Hardcoded JavaScript for 2 posts

```
function showPosts() {
  const div1 = document.getElementById('JSEntry1');
  const div2 = document.getElementById('JSEntry2');

  fetchJSON("/api/v1/posts/1/")
    .then(post => {
      updateDOM(post, div1);
    });
  fetchJSON("/api/v1/posts/2/")
    .then(post => {
      updateDOM(post, div2);
  });
}
```

Simplification: fetchJSON()

- Non-idiomatic fetch wrapper function
- Non-idiomatic because it hides the Promise chain

```
function fetchJSON(url) {
  // Asynchronous GET request to url,
  // decode JSON, and return Promise.
  return fetch(url)
    .then(response => {
      return response.json();
    });
}
```

Promise chain refactor

- Idiomatic Promise chain

```
fetch(url)
  .then(response => {
    return response.json();
  })
  .then(post => {
    updateDOM(post, div);
  });
```

- Explicit Promise variables

```
const p1 = fetch(url);
const p2 = p1.then(response => {
  return response.json();
});
const p3 = p2.then(post => {
  updateDOM(post, div);
});
```

Loop

- We'll hardcode the URL list
  - In a real app, get URL list from REST API

```
function showPosts() {
  const urls = [
    "/api/v1/posts/1/",
    "/api/v1/posts/2/",
  ]
  // ...
  urls.forEach((url) => {
    // ...
  }
}
```

Single JavaScript entry point

- Avoid hardcoding number of posts in HTML and JavaScript

```
<html>
   <div id="JSEntry">Loading posts ...</div>
   <!-- ... -->
</html>
```

```
function showPosts() {
  const urls = [ /*...*/ ];
  const entry = document.getElementById('JSEntry');
  urls.forEach((url) => {
    // ...
  }
}
```

JavaScript creates placeholders

- Create placeholder div nodes in JavaScript loop

```
function showPosts() {
  const urls = [/* ... */];
  const entry = document.getElementById('JSEntry');

  urls.forEach((url) => {
    const div = document.createElement('div');
    entry.appendChild(div);

    // ...
  }
}
```

Native fetch API

- Use the native JavaScript fetch API

```
function showPosts() {
  const urls = [/* ... */];
  const entry = document.getElementById('JSEntry');

  urls.forEach((url) => {
    const div = document.createElement('div');
    entry.appendChild(div);

    fetch(url)
      .then(response => {
        return response.json();
      })
      .then(post => {
        updateDOM(post, div);
      });
  }
}
```

Chained promises

- Chained promises form a sequence of asynchronous events

1. Send GET request
2. Parse JSON
3. Update DOM

```
fetch(url)
  .then(response => {
    return response.json();
  })
  .then(post => {
    updateDOM(post, div);
  });
```

Chained promises

- Fulfilled value of Promise `p1` is the input to handler provided to `p1.then()`
- Fulfilled value of Promise `p2` is the input to handler provided to `p2.then()`
- `Promise p3` **resolves immediately** because `updateDOM` doesn't return a Promise
  - Like the null pointer at the end of a linked list

```
const p1 = fetch(url);
const p2 = p1.then(response => {
  return response.json();
});
const p3 = p2.then(post => {
  updateDOM(post, div);
});
```

Fetch API

```
const p1 = fetch(url);
const p2 = p1.then(response => {
  return response.json();
});
const p3 = p2.then(post => {
  updateDOM(post, div);
});
```

- `fetch()` is an API provided by the browser
- `fetch()` returns a Promise
- Fulfilled value is a Response
  - Fulfilled when when HTTP headers arrive
- Handler executes with fulfilled value Response as input

Response.json()

```
const p1 = fetch(url);
const p2 = p1.then(response => {
  return response.json();
});
const p3 = p2.then(post => {
  updateDOM(post, div);
});
```

- `json()` is a method of `Response`
- `json()` returns a Promise
- Fulfilled value is an `object`
- Fulfilled when HTTP Response body arrives and JSON is parsed
- Handler executes with fulfilled value `object` as input
- `object` is a post from the REST API in this example

Promise chain diagram

```
const p1 = fetch(url);
const p2 = p1.then(response => {
  return response.json();
});
const p3 = p2.then(post => {
  updateDOM(post, div);
});
```

- Imagine a Promise chain as a linked list of function objects

Promise chain diagram

Summary

- *Synchronous programming:* one task at a time
- Problem: synchronous requests are slow
  - Can't send the second request until the first response has arrived
- Solution: asynchronous requests
  - Send requests right away without waiting for the response
- *Asynchronous programming:* interleave tasks

Summary

- A Promise represents a value that will be available in the future
- A Promise controls the flow of asynchronous operations

- Placeholders create an ordered page from unordered asynchronous responses

- Chained promises form a sequence of asynchronous events